

Elementary Number Theory

Elementary Number Theory

Author

MUHO CHOI

Email: sihyeon8240@gmail.com

GitHub: <https://github.com/sihyeon8240>

Source Code & Contributions

GitHub: <https://github.com/sihyeon8240/math>

Feel free to open issues or pull requests for corrections.

© 2026. MUHO CHOI.

This textbook is licensed under a [Creative Commons Attribution 4.0 International License](https://creativecommons.org/licenses/by/4.0/) (CC BY 4.0).

You are free to: Share, copy, and redistribute the material in any medium or format; adapt, remix, transform, and build upon the material for any purpose, even commercially.

Under the following terms: You must give appropriate credit, provide a link to the license, and indicate if changes were made.

Contents

Preface	iv
1 Divisibility Theory in the Integers	1
1.1 The Division Algorithm	1
1.2 The Greatest Common Divisor	4
1.3 The Euclidean Algorithm	16
1.4 The Diophantine Equation $ax+by=c$	25
2 Primes and Their Distribution	26
2.1 The Fundamental Theorem of Arithmetic	26
2.2 The Sieve of Eratosthenes	26
2.3 The Goldbach Conjecture	26
3 The Theory of Congruences	27
3.1 Basic Properties of Congruence	27
3.2 Binary and Decimal Representations of Integers	27
3.3 Linear Congruences and the Chinese Remainder Theorem	27
4 Fermat's Theorem	28
4.1 Fermat's Little Theorem and Pseudoprimes	28
4.2 Wilson's Theorem	28
4.3 The Fermat-Kraitchik Factorization Method	28
5 Number-Theoretic Functions	29
5.1 The Sum and Number of Divisors	29
5.2 The Möbius Inversion Formula	29
5.3 The Greatest Integer Function	29
5.4 An Application to the Calendar	29
6 Euler's Generalization of Fermat's Theorem	30
6.1 Euler's Phi-Function	30
6.2 Euler's Theorem	30
6.3 Some Properties of the Phi-Function	30

7	Primitive Roots and Indices	31
7.1	The Order of an Integer Modulo n	31
7.2	Primitive Roots for Primes	31
7.3	Composite Numbers Having Primitive Roots	31
7.4	The Theory of Indices	31
8	The Quadratic Reciprocity Law	32
8.1	Euler's Criterion	32
8.2	The Legendre Symbol and Its Properties	32
8.3	Quadratic Reciprocity	32
8.4	Quadratic Congruences with Composite Moduli	32
9	Introduction to Cryptography	33
9.1	From Caesar Cipher to Public Key Cryptography	33
9.2	The Knapsack Cryptosystem	33
9.3	An Application of Primitive Roots to Cryptography	33
10	Numbers of Special Form	34
10.1	Perfect Numbers	34
10.2	Mersenne Primes and Amicable Numbers	34
10.3	Fermat Numbers	34
11	Certain Nonlinear Diophantine Equations	35
11.1	The Equation $x^2 + y^2 = z^2$	35
11.2	Fermat's Last Theorem	35
12	Representation of Integers as Sums of Squares	36
12.1	Sums of Two Squares	36
12.2	Sums of More Than Two Squares	36
13	Fibonacci Numbers	37
13.1	The Fibonacci Sequence	37
13.2	Certain Identities Involving Fibonacci Numbers	37
14	Continued Fractions	38
14.1	Finite Continued Fractions	38
14.2	Infinite Continued Fractions	38
14.3	Farey Fractions	38
14.4	Pell's Equation	38
15	Some Modern Developments	39
15.1	Primality Testing and Factorization	39
15.2	An Application to Factoring: Remote Coin Flipping	39
15.3	The Prime Number Theorem and Zeta Function	39

Preface

This textbook uses Lean 4 to formalize selected mathematical statements and proofs.

- **Tools.** Lean is an interactive theorem prover; Mathlib is a library of formalized mathematics. This edition uses Lean 4.33.1 and Mathlib v4.33.1 (revision [0df444a360ea](#)).
- **Reading the proofs.** Each paired statement or proof step gives a mathematical explanation followed by its Lean code. The explanations can be read independently; code fragments are read in order within the complete declaration.
- **Verification.** Lean checks formal proofs under their stated assumptions. Verification covers only statements linked to checked declarations. Agreement between the explanations, displayed code, and checked proofs is reviewed separately.

Chapter 1

Divisibility Theory in the Integers

1.1 The Division Algorithm

Theorem 1.1.1. *Let $a \in \mathbb{Z}$ and $b \in \mathbb{N}^+$. There is a unique pair $(q, r) \in \mathbb{Z} \times \mathbb{N}$ such that*

$$a = qb + r, \quad r < b.$$

*We call q the **quotient** and r the **remainder**.*

```
theorem existsUnique_quotient_remainder (a : ℤ) (b : ℕ+) :  
  ∃! qr : ℤ × ℕ,  
    a = qr.1 * b + qr.2 ∧ qr.2 < b
```

Proof. Since $b \geq 1$,

$$a - (-|a|)b = a + |a|b \geq a + |a| \geq 0.$$

```
classical  
  
have hb : (1 : ℤ) ≤ (b : ℤ) := by  
  exact_mod_cast b.pos  
  
have hnonneg : 0 ≤ a - (-|a|) * (b : ℤ) := by  
  have ha : -|a| ≤ a := neg_abs_le a  
  have habs : 0 ≤ |a| := abs_nonneg a  
  nlinarith
```

Define

$$S := \{n \in \mathbb{N} : a - qb = n \text{ for some } q \in \mathbb{Z}\}.$$

By the preceding inequality, $S \neq \emptyset$. Let $r = \min S$ and choose $q \in \mathbb{Z}$ with $a - qb = r$.

```

let h :  $\exists n : \mathbb{N}, \exists q : \mathbb{Z}, a - q * b = (n : \mathbb{Z})$  :=
  ((a - (-|a|) * b).toNat, -|a|, (Int.toNat_of_nonneg hnonneg).symm)

obtain (q, hq) := Nat.find_spec h

let r :  $\mathbb{N}$  := Nat.find h

have hr0 : (0 :  $\mathbb{Z}$ )  $\leq$  r := Int.natCast_nonneg _

```

By minimality, for $s \in \mathbb{Z}$,

$$0 \leq s, \quad s = a - xb \ (x \in \mathbb{Z}) \implies r \leq s.$$

```

have hminimal :
   $\forall s : \mathbb{Z}, 0 \leq s \rightarrow (\exists x : \mathbb{Z}, a - x * b = s) \rightarrow (r : \mathbb{Z}) \leq s$  := by
  intro s hs (x, hx)

have hn : Nat.find h  $\leq$  s.toNat :=
  Nat.find_min' h (x, hx.trans (Int.toNat_of_nonneg hs).symm)

have hc : (Nat.find h :  $\mathbb{Z}$ )  $\leq$  (s.toNat :  $\mathbb{Z}$ ) :=
  Int.ofNat_le.mpr hn

  rwa [Int.toNat_of_nonneg hs] at hc

```

If $r \geq b$, then

$$0 \leq r - b = a - (q + 1)b < r,$$

contradicting minimality. Hence $r < b$.

```

have hrb_int : (r :  $\mathbb{Z}$ ) < b := by
  by_contra hnot

have hs :  $0 \leq (r : \mathbb{Z}) - b$  := by
  linarith

have hsmaller := hminimal ((r :  $\mathbb{Z}$ ) - b) hs (q + 1, by
  dsimp [r]
  nlinarith [hq])

  linarith

```

Thus $a = qb + r$ and $0 \leq r < b$, proving existence.

```

have ha : a = q * b + r := by
  dsimp [r]
  linarith [hq]

have hrb : r < (b : ℕ) := by
  exact_mod_cast hrb_int

refine ((q, r), (ha, hrb), ?_)

```

Suppose another pair $(q', r') \in \mathbb{Z} \times \mathbb{N}$ satisfies $a = q'b + r'$ and $0 \leq r' < b$.

```

rintro (q', r') (ha', hr'b)

change a = q' * b + r' at ha'

have hr'0 : (0 : ℤ) ≤ r' := Int.natCast_nonneg _

have hr'b_int : (r' : ℤ) < b := by
  exact_mod_cast hr'b

```

Since $q, q' \in \mathbb{Z}$,

$$\begin{aligned}
 q' < q &\implies q - q' \geq 1 \implies r' = r + (q - q')b \geq b, \\
 q' > q &\implies q' - q \geq 1 \implies r = r' + (q' - q)b \geq b.
 \end{aligned}$$

Both contradict the remainder bounds, so $q' = q$.

```

have hqq : q' = q := by
  rcases lt_trichotomy q' q with hlt | heq | hgt

  · have hstep : q' + 1 ≤ q := hlt
    nlinarith

  · exact heq

  · have hstep : q + 1 ≤ q' := hgt
    nlinarith

```

Then $r' = a - q'b = a - qb = r$, so $(q', r') = (q, r)$, proving uniqueness.


```

have hrr_int : (r' : ℤ) = r := by
  rw [hqq] at ha'
  linarith

have hrr : r' = r := by
  exact_mod_cast hrr_int

exact Prod.ext hqq hrr

```

□

1.2 The Greatest Common Divisor

For integers c, z , define $c \mid z$ to mean that $z = ck$ for some $k \in \mathbb{Z}$. In particular, $0 \mid z$ if and only if $z = 0$.

Theorem 1.2.1. *Suppose $a, b \in \mathbb{Z}$ are not both zero. There is a positive natural number d which divides both a and b , is an integer linear combination of them, and is divisible by every common divisor.*

```

theorem exists_positive_common_divisor (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  ∃ d : ℕ, 0 < d ∧ (d : ℤ) ∣ a ∧ (d : ℤ) ∣ b ∧
    (∃ x y : ℤ, (d : ℤ) = a * x + b * y) ∧
    (∀ c : ℤ, c ∣ a → c ∣ b → c ∣ (d : ℤ))

```

Proof. Since a, b are not both zero, $a^2 + b^2 > 0$. Thus the set of positive natural numbers of the form $ax + by$, with $x, y \in \mathbb{Z}$, is nonempty.

```

classical

have hpos : 0 < a * a + b * b := by
  rcases hab with ha | hb

  · nlinarith [sq_pos_of_ne_zero ha, sq_nonneg b]
  · nlinarith [sq_nonneg a, sq_pos_of_ne_zero hb]

have hS : ∃ n : ℕ, 0 < n ∧ ∃ x y : ℤ, a * x + b * y = (n : ℤ) := by
  refine ((a * a + b * b).toNat, _, a, b, _)

  · omega
  · exact (Int.toNat_of_nonneg (le_of_lt hpos)).symm

```

Let d be its least element and choose $x, y \in \mathbb{Z}$ with $ax + by = d$. Every positive natural number represented by such a combination is at least d .

```

let d : ℕ := Nat.find hS

obtain ⟨hdpos, x, y, hxy⟩ := Nat.find_spec hS

have hminimal (n : ℕ) (hn : 0 < n)
  (hlinear : ∃ u v : ℤ, a * u + b * v = (n : ℤ)) : d ≤ n :=
  Nat.find_min' hS ⟨hn, hlinear⟩

```

Suppose $u, v \in \mathbb{Z}$. Apply the division algorithm to $au + bv$ and the positive divisor d , obtaining $au + bv = qd + r$ with $0 \leq r < d$.

```

have hddiv (u v : ℤ) : (d : ℤ) ∣ a * u + b * v := by
  obtain ⟨⟨q, r⟩, ⟨heq, hr⟩, _⟩ :=
    existsUnique_quotient_remainder (a * u + b * v) ⟨d, hdpos⟩

  change a * u + b * v = q * (d : ℤ) + (r : ℤ) at heq
  change r < d at hr

```

The remainder is again a linear combination: $r = a(u - qx) + b(v - qy)$.

```

have hrlinear : a * (u - q * x) + b * (v - q * y) = (r : ℤ) := by
  calc
    a * (u - q * x) + b * (v - q * y) =
      (a * u + b * v) - q * (a * x + b * y) := by
        ring
    _ = r := by
      rw [hxy]
      linarith [heq]

```

If $r \neq 0$, then $r > 0$ and minimality gives $d \leq r$, contradicting $r < d$. Hence $r = 0$.

```

have hrzero : r = 0 := by
  by_contra hne

  have hle := hminimal r (Nat.pos_of_ne_zero hne)
    ⟨u - q * x, v - q * y, hrlinear⟩

  omega

```

Therefore $d \mid au + bv$. Taking $(u, v) = (1, 0)$ and $(0, 1)$ gives $d \mid a$ and $d \mid b$.

```

refine (q, ?_)
rw [hrzero] at heq
simp only [Nat.cast_zero, add_zero, mul_comm] using heq

have hda : (d : ℤ) | a := by
  simp only [mul_one, mul_zero, add_zero] using hddiv 1 0

have hdb : (d : ℤ) | b := by
  simp only [mul_one, mul_zero, zero_add] using hddiv 0 1

```

Suppose $c \mid a$ and $c \mid b$. Choose $u, v \in \mathbb{Z}$ with $a = cu$ and $b = cv$. Then $d = ax + by = c(ux + vy)$, so $c \mid d$. This proves all the asserted properties.

```

have hcommon (c : ℤ) (hca : c ∣ a) (hcb : c ∣ b) : c ∣ (d : ℤ) := by
  obtain ⟨u, hu⟩ := hca
  obtain ⟨v, hv⟩ := hcb

  refine (u * x + v * y, ?_)
  rw [← hxy, hu, hv]
  ring

exact ⟨d, hdpow, hda, hdb, ⟨x, y, hxy.symm⟩, hcommon⟩

```

□

Definition 1.2.2. For $a, b \in \mathbb{Z}$ not both zero, define $\gcd(a, b)$ to be the positive common divisor constructed above from the least positive linear combination. Define $\gcd(0, 0) = 0$.

```

noncomputable def gcd (a b : ℤ) : ℕ :=
  if hab : a ≠ 0 ∨ b ≠ 0 then
    Classical.choose (exists_positive_common_divisor a b hab)
  else 0

```

Theorem 1.2.3. For integers a, b not both zero, $\gcd(a, b)$ is positive, divides both inputs, is an integer linear combination of them, and is divisible by every common divisor.

```

theorem gcd_spec (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  0 < gcd a b ∧ (gcd a b : ℤ) ∣ a ∧ (gcd a b : ℤ) ∣ b ∧
  (∃ x y : ℤ, (gcd a b : ℤ) = a * x + b * y) ∧
  (∀ c : ℤ, c ∣ a → c ∣ b → c ∣ (gcd a b : ℤ))

```

Proof. The definition selects the number constructed above, together with its established properties.

```
rw [gcd, dif_pos hab]
exact Classical.choose_spec (exists_positive_common_divisor a b hab)
```

□

Theorem 1.2.4. *Suppose $a, b \in \mathbb{Z}$ are not both zero and $c \in \mathbb{Z}$ divides both. Then $c \leq \gcd(a, b)$.*

```
theorem common_divisor_le_gcd (a b c :  $\mathbb{Z}$ ) (hab : a  $\neq$  0  $\vee$  b  $\neq$  0)
  (hca : c  $\mid$  a) (hcb : c  $\mid$  b) : c  $\leq$  (gcd a b :  $\mathbb{Z}$ )
```

Proof. Write $d = \gcd(a, b) > 0$. Since $c \mid d$, there is $k \in \mathbb{Z}$ with $d = ck$. If $c > 0$, positivity of d forces $k > 0$, hence $k \geq 1$ and $d \geq c$. If $c \leq 0$, then $c < d$ immediately.

```
obtain (hpos, _, _, _, hcommon) := gcd_spec a b hab

have hdpos : (0 :  $\mathbb{Z}$ ) < gcd a b := by
  exact_mod_cast hpos

obtain (k, hk) := hcommon c hca hcb

by_cases hc : 0 < c

· have hkpos : 0 < k := by
  nlinarith

  have hkone : 1  $\leq$  k := hkpos

  nlinarith

· linarith
```

□

Theorem 1.2.5. *The zero-pair convention is $\gcd(0, 0) = 0$.*

```
theorem gcd_zero_zero : gcd 0 0 = 0
```

Proof. Neither input is nonzero, so the definition assigns the value zero. This is a convention; the common divisors of zero and zero have no greatest integer.

```

unfold gcd
rw [dif_neg (by
  rintro (h | h)
  exact h rfl
  exact h rfl)]

```

□

Theorem 1.2.6. *Suppose $a, b \in \mathbb{Z}$ are not both zero and $d \in \mathbb{N}$ divides both. If every integer common divisor is at most d , then $\gcd(a, b) = d$.*

```

theorem gcd_eq_of_greatest (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) (d : ℕ)
  (hda : (d : ℤ) ∣ a) (hdb : (d : ℤ) ∣ b)
  (hgreatest : ∀ c : ℤ, c ∣ a → c ∣ b → c ≤ (d : ℤ)) : gcd a b = d

```

Proof. Since $\gcd(a, b)$ is a common divisor, the hypothesis gives $\gcd(a, b) \leq d$. The preceding bound applied to d gives $d \leq \gcd(a, b)$. Antisymmetry proves equality.

```

have hspec := gcd_spec a b hab
have hle := hgreatest (gcd a b) hspec.2.1 hspec.2.2.1
have hge := common_divisor_le_gcd a b d hab hda hdb
exact_mod_cast (le_antisymm hle hge)

```

□

Theorem 1.2.7. *Suppose $a, b \in \mathbb{Z}$ are not both zero. There are integers x, y such that $\gcd(a, b) = ax + by$.*

```

theorem exists_gcd_eq_mul_add_mul (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  ∃ x y : ℤ, (gcd a b : ℤ) = a * x + b * y

```

Proof. The linear-combination representation is one of the properties established when constructing the gcd.

```

exact (gcd_spec a b hab).2.2.2.1

```

□

Theorem 1.2.8. Suppose $a, b \in \mathbb{Z}$ are not both zero. The set of integer linear combinations $ax + by$ is exactly the set of integer multiples of $\gcd(a, b)$.

```
theorem linear_combinations_eq_gcd_multiples (a b :  $\mathbb{Z}$ ) (hab : a  $\neq$  0  $\vee$  b
 $\hookrightarrow$   $\neq$  0) :
  {z :  $\mathbb{Z}$  |  $\exists$  x y :  $\mathbb{Z}$ , z = a * x + b * y} =
  {z :  $\mathbb{Z}$  |  $\exists$  n :  $\mathbb{Z}$ , z = n * (gcd a b :  $\mathbb{Z}$ )}
```

Proof. Write $d = \gcd(a, b) = ax_0 + by_0$. Since d divides both inputs, choose $u, v \in \mathbb{Z}$ with $a = du$ and $b = dv$. We prove both set inclusions.

```
obtain (x0, y0, hbezout) := exists_gcd_eq_mul_add_mul a b hab

have hcommon := (gcd_spec a b hab).2

obtain (u, hu) := hcommon.1
obtain (v, hv) := hcommon.2.1

apply Set.ext
intro z
constructor
```

If $z = ax + by$, then $z = (ux + vy)d$, so z is an integer multiple of d .

```
· rintro (x, y, hz)

refine (u * x + v * y, ?_)
calc
  z = a * x + b * y := hz
  _ = ((gcd a b :  $\mathbb{Z}$ ) * u) * x + ((gcd a b :  $\mathbb{Z}$ ) * v) * y := by
    rw [← hu, ← hv]
  _ = (u * x + v * y) * (gcd a b :  $\mathbb{Z}$ ) := by
    ring
```

Conversely, if $z = nd$, then $z = a(nx_0) + b(ny_0)$, an integer linear combination of a, b .

```
· rintro (n, hz)

refine (n * x0, n * y0, ?_)
rw [hz, hbezout]
ring
```

□

Theorem 1.2.9. *Let $a, b \in \mathbb{Z}$ be not both zero. They are relatively prime, meaning $\gcd(a, b) = 1$, if and only if there are integers x, y with $1 = ax + by$.*

```
theorem gcd_eq_one_iff_exists_mul_add_mul (a b : ℤ) (hab : a ≠ 0 ∨ b ≠
↪ 0) :
  gcd a b = 1 ↔ ∃ x y : ℤ, 1 = a * x + b * y
```

Proof. For the forward implication, substitute $\gcd(a, b) = 1$ in the previously proved Bezout identity.

```
constructor

· intro h

obtain ⟨x, y, hxy⟩ := exists_gcd_eq_mul_add_mul a b hab

exact ⟨x, y, by simp only [h, Nat.cast_one] using hxy⟩
```

Conversely, 1 divides both inputs. If c is a common divisor, write $a = cu$ and $b = cv$. Then $1 = c(ux + vy)$. For $c > 0$ the second factor is a positive integer, hence at least one, so $c \leq 1$; for $c \leq 0$ the same bound is immediate. The order characterization of the gcd gives the result.

```
· rintro ⟨x, y, hxy⟩
  apply gcd_eq_of_greatest a b hab 1

· exact ⟨a, by ring⟩
· exact ⟨b, by ring⟩
· intro c hca hcb

obtain ⟨u, hu⟩ := hca
obtain ⟨v, hv⟩ := hcb

have hone : 1 = c * (u * x + v * y) := by
  calc
    1 = a * x + b * y := hxy
  _ = c * (u * x + v * y) := by
    rw [hu, hv]
    ring

by_cases hc : 0 < c

· have hk : 0 < u * x + v * y := by
  nlinarith
```

```

have hkone : 1 ≤ u * x + v * y := hk

norm_num only [Nat.cast_one]
nlinarith

· norm_num only [Nat.cast_one]
  omega

```

□

Corollary 1.2.10. *For integers a, b not both zero, set $d = \gcd(a, b)$. Then $\gcd(a/d, b/d) = 1$, where the quotients are integers.*

```

theorem gcd_div_gcd_eq_one (a b : ℤ) (hab : a ≠ 0 ∨ b ≠ 0) :
  gcd (a / (gcd a b : ℤ)) (b / (gcd a b : ℤ)) = 1

```

Proof. Since the positive gcd divides both inputs, write $a = du$ and $b = dv$. The integer quotients are exactly u and v , since $d \neq 0$.

```

obtain ⟨hdpos, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ := gcd_spec a b hab

generalize hg : gcd a b = d at *

have hdne : (d : ℤ) ≠ 0 := by
  exact_mod_cast (ne_of_gt hdpos)

have ha : a / (d : ℤ) = u := by
  rw [hu]
  exact Int.mul_ediv_cancel_left u hdne

have hb : b / (d : ℤ) = v := by
  rw [hv]
  exact Int.mul_ediv_cancel_left v hdne

```

Choose Bezout coefficients with $d = ax + by$. Substitution gives $d = d(ux + vy)$, and cancellation of the nonzero integer d yields $1 = ux + vy$.


```

have hone : (1 : ℤ) = u * x + v * y := by
  apply mul_left_cancel₀ hdne
calc
  (d : ℤ) * 1 = a * x + b * y := by
    simp using hxy
  _ = (d : ℤ) * (u * x + v * y) := by
    rw [hu, hv]
    ring

```

The identity rules out $u = v = 0$. Apply the preceding characterization to conclude that the two quotients are relatively prime.

```

have huv : u ≠ 0 v v ≠ 0 := by
  by_contra h
  push Not at h

obtain (rfl, rfl) := h

norm_num at hone

rw [ha, hb]
exact (gcd_eq_one_iff_exists_mul_add_mul u v huv).2 ⟨x, y, hone⟩

```

□

Corollary 1.2.11. *Let $a, b, c \in \mathbb{Z}$. If $\gcd(a, b) = 1$, $a \mid c$, and $b \mid c$, then $ab \mid c$.*

```

theorem mul_dvd_of_coprime (a b c : ℤ) (hab : gcd a b = 1)
  (hac : a ∣ c) (hbc : b ∣ c) : a * b ∣ c

```

Proof. The inputs cannot both be zero because $\gcd(0, 0) = 0$. Thus the coprimality characterization supplies integers x, y with $1 = ax + by$.

```

have hnz : a ≠ 0 v b ≠ 0 := by
  by_contra h
  push Not at h

obtain (rfl, rfl) := h

rw [gcd_zero_zero] at hab
contradiction

obtain ⟨x, y, hxy⟩ := (gcd_eq_one_iff_exists_mul_add_mul a b hnz).1 hab

```

Write $c = ar = bs$. Multiplying the Bezout identity by c and using these two expressions gives $c = acx + bcy = ab(sx + ry)$. The integer $sx + ry$ is the required divisibility witness.

```

obtain ⟨r, hr⟩ := hac

obtain ⟨s, hs⟩ := hbc

refine ⟨s * x + r * y, ?_⟩
calc
  c = c * (a * x + b * y) := by
    rw [← hxy]
    ring
  _ = a * c * x + b * c * y := by
    ring
  _ = a * (b * s) * x + b * (a * r) * y := by
    nth_rw 1 [hs]
    rw [hr]
  _ = (a * b) * (s * x + r * y) := by
    ring

```

□

Theorem 1.2.12. *Let $a, b, c \in \mathbb{Z}$ with $\gcd(a, b) = 1$. If $a \mid bc$, then $a \mid c$ (Euclid's lemma).*

```

theorem dvd_of_dvd_mul_of_coprime (a b c : ℤ) (hab : gcd a b = 1)
  (hdiv : a ∣ b * c) : a ∣ c

```

Proof. As before, coprimality excludes the zero pair, so choose integers x, y with $1 = ax + by$.

```

have hnz : a ≠ 0 ∨ b ≠ 0 := by
  by_contra h
  push Not at h

obtain ⟨rfl, rfl⟩ := h

  rw [gcd_zero_zero] at hab
  contradiction

obtain ⟨x, y, hxy⟩ := (gcd_eq_one_iff_exists_mul_add_mul a b hnz).1 hab

```

Write $bc = ak$. Multiplication by c gives $c = acx + bcy = a(cx + ky)$, which exhibits an integer witness for $a \mid c$.

```

obtain ⟨k, hk⟩ := hdiv

refine ⟨c * x + k * y, ?_⟩
calc
  c = (a * x + b * y) * c := by
    rw [← hxy]
    ring
  _ = a * (c * x) + (b * c) * y := by
    ring
  _ = a * (c * x + k * y) := by
    rw [hk]
    ring

```

□

Theorem 1.2.13. *Let $a, b \in \mathbb{Z}$ be not both zero and let $d \in \mathbb{Z}$ be positive. Then $d = \gcd(a, b)$ if and only if d divides both inputs and every integer common divisor of a, b divides d .*

```

theorem eq_gcd_iff_common_divisor (a b d :  $\mathbb{Z}$ ) (hab :  $a \neq 0 \vee b \neq 0$ )
  (hd :  $0 < d$ ) :
  d = (gcd a b :  $\mathbb{Z}$ ) ↔
    d | a ∧ d | b ∧ (∀ c :  $\mathbb{Z}$ , c | a → c | b → c | d)

```

Proof. If $d = \gcd(a, b)$, all three divisibility properties follow from the construction of the gcd.

```

constructor

· intro heq
  subst d

obtain ⟨_, hda, hdb, _, hcommon⟩ := gcd_spec a b hab

exact ⟨hda, hdb, hcommon⟩

```

Conversely, let $g = \gcd(a, b) > 0$. Because d is a common divisor, $d \leq g$. Since g divides both inputs, the last hypothesis gives $d = gk$. Positivity forces the integer k to be at least one, so $g \leq d$. The two bounds imply $d = g$.

```

· rintro (hda, hdb, hcommon)

have hle := common_divisor_le_gcd a b d hab hda hdb

obtain (hgpos, hga, hgb, _, _) := gcd_spec a b hab
obtain (k, hk) := hcommon (gcd a b) hga hgb

have hgpos' : (0 : ℤ) < gcd a b := by
  exact_mod_cast hgpos

have hkpos : 0 < k := by
  nlinarith

have hkone : 1 ≤ k := hkpos

have hge : (gcd a b : ℤ) ≤ d := by
  nlinarith

exact le_antisymm hle hge

```

□

Lemma 1.2.14. *For all integers a, b , the nonnegative gcd constructed above agrees with Mathlib's standard integer gcd, including $(a, b) = (0, 0)$.*

```

theorem gcd_eq_int_gcd (a b : ℤ) : gcd a b = Int.gcd a b

```

Proof. If the inputs are not both zero, each gcd divides the other by the common-divisor property. Since both are natural numbers, they are equal.

```

by_cases hab : a ≠ 0 ∨ b ≠ 0

· obtain (_, hda, hdb, _, hgreatest) := gcd_spec a b hab
  apply Nat.dvd_antisymm

  · exact Int.dvd_gcd hda hdb
  · exact Int.natCast_dvd_natCast.mp
    (hgreatest (Int.gcd a b) (Int.gcd_dvd_left a b) (Int.gcd_dvd_right
      ↪ a b))

```

For the zero pair, both definitions give zero. This comparison identifies the representations after the local gcd properties have been established; none of those proofs uses the comparison.

```

• have ha : a = 0 := Classical.byContradiction (fun h => hab (Or.inl h))
  have hb : b = 0 := Classical.byContradiction (fun h => hab (Or.inr h))
  rw [ha, hb, gcd_zero_zero, Int.gcd_zero_left]
  rfl

```

□

1.3 The Euclidean Algorithm

We retain the nonnegative gcd, with $\text{gcd}(0, 0) = 0$. Pairs with the same common divisors have equal gcds: for nonzero pairs, each gcd is at most the other; a zero pair is characterized by having zero as a common divisor.

Lemma 1.3.1. *If $a, b, q, r \in \mathbb{Z}$ and $a = qb + r$, then $\text{gcd}(a, b) = \text{gcd}(b, r)$. No sign or size condition on the remainder is needed for this identity.*

```

theorem gcd_eq_gcd_remainder (a b q r : ℤ) (ha : a = q * b + r) :
  gcd a b = gcd b r

```

Proof. It suffices to compare common divisors. If $a = cu$ and $b = cv$, then $r = c(u - qv)$. Thus a common divisor of a, b also divides b, r .

```

apply gcd_eq_of_common_divisors
intro c
constructor

• rintro ⟨⟨u, hu⟩, ⟨v, hv⟩⟩
  refine ⟨⟨v, hv⟩, ⟨u - q * v, ?_⟩⟩
  calc
    r = a - q * b := by
      linarith [ha]
    _ = c * (u - q * v) := by
      rw [hu, hv]
      ring

```

Conversely, if $b = cu$ and $r = cv$, then $a = c(qu + v)$. Both implications use explicit integer witnesses.

```

• rintro ⟨⟨u, hu⟩, ⟨v, hv⟩⟩
  exact ⟨⟨q * u + v, by
    rw [ha, hu, hv]
    ring⟩, ⟨u, hu⟩⟩

```

□

Lemma 1.3.2. *For all integers a, b , taking absolute values leaves the gcd unchanged: $\gcd(|a|, |b|) = \gcd(a, b)$.*

```
theorem gcd_abs (a b : ℤ) : gcd |a| |b| = gcd a b
```

Proof. An integer c divides $|z|$ exactly when it divides z . For $z \geq 0$ this is immediate; for $z < 0$, negate the multiplication witness. Apply this to both inputs and use the common-divisor characterization.

```
have hdiv (c z : ℤ) : c | |z| ↔ c | z := by
  by_cases hz : 0 ≤ z

  · rw [abs_of_nonneg hz]
  · rw [abs_of_neg (lt_of_not_ge hz)]
    constructor <,> rintro ⟨k, hk⟩ <,> exact ⟨-k, by linarith⟩

exact gcd_eq_of_common_divisors _ _ _ _ (fun c => by rw [hdiv, hdiv])
```

□

Absolute values reduce the computation to nonnegative inputs. No initial ordering is necessary: dividing a smaller input by a larger one simply gives quotient zero. A finite division trace records every quotient and remainder. Its terminal pair is $(g, 0)$; each preceding pair (a, b) satisfies $a = qb + r$ with $0 \leq r < b$, and is followed by (b, r) .

Definition 1.3.3. The relation $\text{EuclideanTrace } a \ b \ g$ records such a finite computation. The natural number types encode nonnegativity, and $r < b$ records strict decrease.

```
inductive EuclideanTrace : ℕ → ℕ → ℕ → Prop
| done (a : ℕ) : EuclideanTrace a 0 a
| step (a b r g : ℕ) (q : ℤ) (hr : r < b)
  (ha : (a : ℤ) = q * (b : ℤ) + (r : ℤ))
  (tail : EuclideanTrace b r g) : EuclideanTrace a b g
```

Theorem 1.3.4. *For every $a, b \in \mathbb{N}$, a finite Euclidean division trace starts at (a, b) and ends at some $(g, 0)$.*

```
theorem euclideanTrace_exists (a b : ℕ) : ∃ g, EuclideanTrace a b g
```

Proof. Use strong induction on the second input, allowing the first input to vary. If the second input is zero, the computation has already ended.

```

induction b using Nat.strong_induction_on generalizing a with
| h b ih =>
  by_cases hb : b = 0

  · subst b
    exact ⟨a, EuclideanTrace.done a⟩

```

Otherwise the previously proved division algorithm supplies $a = qb + r$ with $r < b$. The induction hypothesis gives a finite trace from (b, r) ; prefix the recorded division to obtain the required trace.

```

· obtain ⟨⟨q, r⟩, ⟨ha, hr⟩, _⟩ :=
  existsUnique_quotient_remainder (a : ℤ) (b, Nat.pos_of_ne_zero hb)

obtain ⟨g, hg⟩ := ih r hr b

exact ⟨g, EuclideanTrace.step a b r g q hr ha hg⟩

```

□

Theorem 1.3.5. *If a Euclidean trace starts at (a, b) and ends at $(g, 0)$, then $\gcd(a, b) = g$.*

```

theorem EuclideanTrace.gcd_eq {a b g : ℕ} (h : EuclideanTrace a b g) :
  gcd (a : ℤ) (b : ℤ) = g

```

Proof. Induct on the finite trace. At the terminal pair, $\gcd(a, 0) = a$ for $a \in \mathbb{N}$, by greatestness (and the zero-pair convention). At each earlier division, the remainder lemma preserves the gcd, so the induction hypothesis gives the same terminal value.

```

induction h with
| done a => exact gcd_zero_right a
| step a b r g q hr ha tail ih =>
  exact (gcd_eq_gcd_remainder _ _ q _ ha).trans ih

```

□

Corollary 1.3.6. *If the initial second input b is positive, then the terminal entry g of a Euclidean trace is positive. Hence it is the last nonzero remainder, or the initial divisor if the first remainder is zero.*

```
theorem EuclideanTrace.positive {a b g : ℕ} (h : EuclideanTrace a b g)
  (hb : 0 < b) : 0 < g
```

Proof. The terminal entry equals $\gcd(a, b)$. Since $b > 0$, the inputs are not both zero, and positivity follows from the local gcd specification.

```
rw [← h.gcd_eq]
exact (gcd_spec a b (0r.inr (by exact_mod_cast (ne_of_gt hb)))).1
```

□

Theorem 1.3.7. For integers a, b, k with $k > 0$, we have $\gcd(ka, kb) = k \gcd(a, b)$.

```
theorem gcd_mul_of_pos (a b k : ℤ) (hk : 0 < k) :
  (gcd (k * a) (k * b) : ℤ) = k * (gcd a b : ℤ)
```

Proof. First assume a, b are not both zero. Write $d = \gcd(a, b)$, $a = du$, $b = dv$, and $d = ax + by$. The scaled pair is not both zero, and $kd > 0$.

```
by_cases hab : a ≠ 0 ∨ b ≠ 0
· obtain ⟨hdpos, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ := gcd_spec a b hab
  have hpair : k * a ≠ 0 ∨ k * b ≠ 0 := by
    rcases hab with ha | hb
    · exact 0r.inl (mul_ne_zero (ne_of_gt hk) ha)
    · exact 0r.inr (mul_ne_zero (ne_of_gt hk) hb)
  have hdpos' : (0 : ℤ) < gcd a b := by
    exact_mod_cast hdpos
```

The witnesses u, v show that kd divides both scaled inputs. If c divides them with $ka = cs$ and $kb = ct$, then $kd = k(ax + by) = c(sx + ty)$, so every common divisor divides kd . The local divisibility characterization identifies kd as their gcd.


```

apply Eq.symm
apply (eq_gcd_iff_common_divisor _ _ _ hpair (mul_pos hk hdpow')).2
refine ⟨⟨u, by
  nth_rw 1 [hu]
  ring⟩, ⟨v, by
  nth_rw 1 [hv]
  ring⟩, ?_⟩
rintro c ⟨s, hs⟩ ⟨t, ht⟩
refine ⟨s * x + t * y, ?_⟩
calc
  k * (gcd a b : ℤ) = (k * a) * x + (k * b) * y := by
    rw [hxy]
    ring
  _ = c * (s * x + t * y) := by
    rw [hs, ht]
    ring

```

If both original inputs are zero, both sides vanish by the zero-pair convention.

```

· push Not at hab

obtain ⟨rfl, rfl⟩ := hab

simp [gcd_zero_zero]

```

□

Corollary 1.3.8. *For integers a, b and any nonzero integer k , we have $\gcd(ka, kb) = |k| \gcd(a, b)$.*

```

theorem gcd_mul_of_ne_zero (a b k : ℤ) (hk : k ≠ 0) :
  (gcd (k * a) (k * b) : ℤ) = |k| * (gcd a b : ℤ)

```

Proof. Take absolute values of the scaled inputs, use $|kz| = |k||z|$, and apply the positive-factor result to $|k| > 0$. Finally remove the absolute values from the original gcd.

```

calc
  (gcd (k * a) (k * b) : ℤ) = gcd (|k| * |a|) (|k| * |b|) := by
    rw [← gcd_abs (k * a) (k * b), abs_mul, abs_mul]
  _ = |k| * (gcd |a| |b| : ℤ) := gcd_mul_of_pos _ _ _ (abs_pos.mpr hk)
  _ = |k| * (gcd a b : ℤ) := by
    rw [gcd_abs]

```

□

Lemma 1.3.9. *Any two nonzero integers a, b have a positive natural number that is divisible by both.*

```
theorem exists_positive_common_multiple (a b : ℤ) (ha : a ≠ 0) (hb : b ≠
  0) :
  ∃ m : ℕ, 0 < m ∧ a ∣ (m : ℤ) ∧ b ∣ (m : ℤ)
```

Proof. The integer $(ab)^2$ is positive. Its natural-number representation has the same value, and $(ab)^2 = a(ab^2) = b(a^2b)$ supplies the two divisibility witnesses.

```
have hp : 0 < (a * b) * (a * b) := mul_self_pos.mpr (mul_ne_zero ha hb)

refine ((a * b) * (a * b)).toNat, ?_, ?_, ?_

· omega
· rw [Int.toNat_of_nonneg (le_of_lt hp)]
  exact (a * b * b, by ring)

· rw [Int.toNat_of_nonneg (le_of_lt hp)]
  exact (a * a * b, by ring)
```

□

Definition 1.3.10. For nonzero integers a, b , define $\text{lcm}(a, b)$ to be the least positive common multiple, whose existence was just proved. Extend the definition by zero when either input is zero.

```
noncomputable def lcm (a b : ℤ) : ℕ :=
  if h : a ≠ 0 ∧ b ≠ 0 then
    Nat.find (exists_positive_common_multiple a b h.1 h.2)
  else 0
```

Theorem 1.3.11. *For nonzero integers a, b , the number $m = \text{lcm}(a, b)$ is positive, satisfies $a \mid m$ and $b \mid m$, and obeys $m \leq c$ for every positive integer common multiple c .*

```
theorem lcm_spec (a b : ℤ) (ha : a ≠ 0) (hb : b ≠ 0) :
  0 < lcm a b ∧ a ∣ (lcm a b : ℤ) ∧ b ∣ (lcm a b : ℤ) ∧
  (∀ c : ℤ, a ∣ c → b ∣ c → 0 < c → (lcm a b : ℤ) ≤ c)
```

Proof. The least-element construction supplies positivity and both divisibility properties.

```

classical

rw [lcm, dif_pos (ha, hb)]

obtain ⟨hp, hma, hmb⟩ := Nat.find_spec (exists_positive_common_multiple
  ↪ a b ha hb)

refine ⟨hp, hma, hmb, ?_⟩

```

A positive integer common multiple c is also a positive natural number with the same value. Minimality among natural common multiples gives $m \leq c$, after converting back to integers.

```

intro c hac hbc hc

have hc' : (c.toNat : ℤ) = c := Int.toNat_of_nonneg (le_of_lt hc)

have hmin := Nat.find_min' (exists_positive_common_multiple a b ha hb)
  (show 0 < c.toNat ∧ a | (c.toNat : ℤ) ∧ b | (c.toNat : ℤ) from
    (by omega, by rwa [hc'], by rwa [hc']))

exact_mod_cast (show (Nat.find (exists_positive_common_multiple a b ha
  ↪ hb) : ℤ) ≤ c by
  exact (Int.ofNat_le.mpr hmin).trans_eq hc'))

```

□

Theorem 1.3.12. *For positive integers a, b , we have $\gcd(a, b) \operatorname{lcm}(a, b) = ab$.*

```

theorem gcd_mul_lcm (a b : ℤ) (ha : 0 < a) (hb : 0 < b) :
  (gcd a b : ℤ) * (lcm a b : ℤ) = a * b

```

Proof. Put $d = \gcd(a, b) > 0$. Choose $u, v, x, y \in \mathbb{Z}$ such that $a = du, b = dv$, and $d = ax + by$. Since $b > 0$, we have $v > 0$. The positive integer av is a common multiple because $av = bu$.

```

obtain ⟨hd, ⟨u, hu⟩, ⟨v, hv⟩, ⟨x, y, hxy⟩, _⟩ :=
  gcd_spec a b (Or.inl (ne_of_gt ha))

generalize hg : gcd a b = d at *

have hd' : (0 : ℤ) < d := by
  exact_mod_cast hd

```

```

have hvpos : 0 < v := by
  nlinarith [hv]

have hmpos : 0 < a * v := mul_pos ha hvpos

have ham : a | a * v := ⟨v, rfl⟩

have hbm : b | a * v := ⟨u, by
  rw [hu, hv]
  ring⟩

```

Every common multiple c is divisible by av . Indeed, write $c = as = bt$ and multiply the Bezout identity by c : $dc = ab(tx + sy) = d av(tx + sy)$. Cancel the nonzero factor d to obtain the required witness.

```

have hdiv (c : ℤ) (hac : a | c) (hbc : b | c) : a * v | c := by
  obtain ⟨s, hs⟩ := hac
  obtain ⟨t, ht⟩ := hbc

  refine ⟨t * x + s * y, ?_⟩
  apply mul_left_cancel₀ (ne_of_gt hd')
  calc
    (d : ℤ) * c = (a * x + b * y) * c := by
      rw [hxy]
    _ = a * c * x + b * c * y := by
      ring
    _ = a * (b * t) * x + b * (a * s) * y := by
      nth_rw 1 [ht]
      rw [hs]
    _ = (d : ℤ) * (a * v * (t * x + s * y)) := by
      rw [hv]
      ring

```

Let $\ell = \text{lcm}(a, b) > 0$. Minimality gives $\ell \leq av$, while the preceding argument gives $\ell = avk$. Positivity forces the integer $k \geq 1$, so $\ell \geq av$. Therefore $\ell = av$, and $d\ell = dav = ab$.

```

obtain ⟨hlpos, hla, hlb, hleast⟩ := lcm_spec a b (ne_of_gt ha) (ne_of_gt hb)

have hle : (lcm a b : ℤ) ≤ a * v := hleast _ ham hbm hmpos

obtain ⟨k, hk⟩ := hdiv (lcm a b) hla hlb

have hlpos' : (0 : ℤ) < lcm a b := by
  exact_mod_cast hlpos

```

```

have hkpos : 0 < k := by
  nlinarith

have hkone : 1 ≤ k := hkpos

have heq : (lcm a b : ℤ) = a * v := by
  nlinarith

rw [heq, hv]
ring

```

□

Lemma 1.3.13. *For all integers a, b , the lcm constructed above agrees with Mathlib's standard integer lcm, including zero inputs.*

```

theorem lcm_eq_int_lcm (a b : ℤ) : lcm a b = Int.lcm a b

```

Proof. If either input is zero, both definitions give zero.

```

by_cases ha : a = 0

· simp only [ha, lcm, ne_eq, not_true_eq_false, false_and, ↓reduceDite,
  Int.lcm_zero_left]

· by_cases hb : b = 0

  · simp only [hb, lcm, ne_eq, not_true_eq_false, and_false,
    ↪ ↓reduceDite,
    Int.lcm_zero_right]

```

Otherwise the standard lcm is a positive common multiple. It divides every positive common multiple c , so it is at most c . The previously proved uniqueness of the least positive common multiple identifies it with our construction.

```

· apply lcm_eq_of_least a b ha hb (Int.lcm a b)
  (Int.lcm_pos ha hb) (Int.dvd_lcm_left a b) (Int.dvd_lcm_right a b)
  intro c hac hbc hc
  apply Int.le_of_dvd hc
  have hc' : (c.toNat : ℤ) = c := Int.toNat_of_nonneg hc.le
  rw [← hc']
  apply Int.natCast_dvd_natCast.mpr

```

```
exact Int.lcm_dvd (by rwa [hc']) (by rwa [hc'])
```

□

1.4 The Diophantine Equation $ax+by=c$

Chapter 2

Primes and Their Distribution

2.1 The Fundamental Theorem of Arithmetic

2.2 The Sieve of Eratosthenes

2.3 The Goldbach Conjecture

Chapter 3

The Theory of Congruences

3.1 Basic Properties of Congruence

3.2 Binary and Decimal Representations of Integers

3.3 Linear Congruences and the Chinese Remainder Theorem

Chapter 4

Fermat's Theorem

4.1 Fermat's Little Theorem and Pseudoprimes

4.2 Wilson's Theorem

4.3 The Fermat-Kraitchik Factorization Method

Chapter 5

Number-Theoretic Functions

5.1 The Sum and Number of Divisors

5.2 The Möbius Inversion Formula

5.3 The Greatest Integer Function

5.4 An Application to the Calendar

Chapter 6

Euler's Generalization of Fermat's Theorem

6.1 Euler's Phi-Function

6.2 Euler's Theorem

6.3 Some Properties of the Phi-Function

Chapter 7

Primitive Roots and Indices

7.1 The Order of an Integer Modulo n

7.2 Primitive Roots for Primes

7.3 Composite Numbers Having Primitive Roots

7.4 The Theory of Indices

Chapter 8

The Quadratic Reciprocity Law

8.1 Euler's Criterion

8.2 The Legendre Symbol and Its Properties

8.3 Quadratic Reciprocity

8.4 Quadratic Congruences with Composite Moduli

Chapter 9

Introduction to Cryptography

9.1 From Caesar Cipher to Public Key Cryptography

9.2 The Knapsack Cryptosystem

9.3 An Application of Primitive Roots to Cryptography

Chapter 10

Numbers of Special Form

10.1 Perfect Numbers

10.2 Mersenne Primes and Amicable Numbers

10.3 Fermat Numbers

Chapter 11

Certain Nonlinear Diophantine Equations

11.1 The Equation $x^2 + y^2 = z^2$

11.2 Fermat's Last Theorem

Chapter 12

Representation of Integers as Sums of Squares

12.1 Sums of Two Squares

12.2 Sums of More Than Two Squares

Chapter 13

Fibonacci Numbers

13.1 The Fibonacci Sequence

13.2 Certain Identities Involving Fibonacci Numbers

Chapter 14

Continued Fractions

14.1 Finite Continued Fractions

14.2 Infinite Continued Fractions

14.3 Farey Fractions

14.4 Pell's Equation

Chapter 15

Some Modern Developments

15.1 Primality Testing and Factorization

15.2 An Application to Factoring: Remote Coin Flipping

15.3 The Prime Number Theorem and Zeta Function

Bibliography

- [1] David M. Burton, *Elementary Number Theory*, McGraw-Hill International Editions.